

AJAX e Fetch — Appunti

1. Cos'è AJAX e perché serve

AJAX (Asynchronous JavaScript and XML, anche se oggi si usa quasi sempre JSON al posto di XML) permette a JavaScript di aggiornare una pagina senza ricaricarla. Nel modello classico, ogni dato nuovo richiede di ricaricare tutta la pagina dal server. Con AJAX la pagina viene scaricata una sola volta, e JavaScript apre da solo connessioni al server solo quando servono dati nuovi, aggiornando poi solo il pezzo di pagina necessario. È così che nascono le RIA (Rich Internet Application) come Gmail, Instagram o Netflix.

```
// Pagina classica: ricarico tutto per ogni dato nuovo.  
// Pagina con AJAX: la pagina resta aperta, solo i dati cambiano.
```

2. Cos'è una API e come funziona

Una API (Application Programming Interface) è il contratto che un server mette a disposizione per farsi chiedere dati o operazioni da un altro programma. Il meccanismo è sempre client-server: il client (il nostro JavaScript) fa una richiesta, il server la riceve, la elabora e manda una risposta. Le API REST, le più comuni, definiscono URL, metodo HTTP, headers, struttura della richiesta e struttura della risposta, quasi sempre in formato JSON.

```
// Client chiede -> Server elabora -> Server risponde (JSON)
```

3. Il problema dell'asincronia

JavaScript è monothread: un solo flusso di esecuzione alla volta. Chiedere dati a un server richiede tempo, e se JavaScript si fermasse ad aspettare la risposta, la pagina si bloccherebbe (niente click, niente scroll). Per questo il browser gestisce le richieste in modo asincrono: lancia la richiesta e continua a fare altro, avvisando JavaScript solo quando la risposta è pronta.

```
const callback = () => { console.log('Arrivata la risposta!'); };  
setTimeout(callback, 1000);  
console.log('Il codice continua subito qui...');  
// Ordine di stampa: prima 'continua subito qui', poi 'Arrivata...'
```

4. La soluzione: async/await

Immagina di ordinare un caffè al bar: il barista lo prepara, e nel frattempo tu puoi fare altro, ma quando dice 'pronto' vai a prenderlo, perché non puoi berlo prima che esista. 'await' fa esattamente questo: aspetta il risultato di un'operazione asincrona prima di proseguire alla riga dopo, senza bloccare il resto della pagina. 'await' si usa solo dentro una funzione marcata 'async'.

```
(async () => {  
  const ordinaCaffè = () => new Promise((resolve) => {  
    setTimeout(() => resolve('Caffè pronto!'), 500);  
  });  
  const risultato = await ordinaCaffè();  
  console.log(risultato);  
})();
```

5. fetch() base: come si usa, cosa restituisce

fetch() è la funzione per fare richieste HTTP verso un server o una API. fetch(url) fa partire la richiesta e restituisce una Promise, quindi va sempre usata con await. Quello che restituisce NON è ancora il dato che ci interessa, ma un oggetto Response, che rappresenta la risposta HTTP grezza (status, headers, e un body ancora da leggere).

```
const response = await fetch('https://api.esempio.it/utenti');
console.log(response); // oggetto Response: status, headers...
```

6. I due await obbligatori

Quasi tutte le API rispondono in JSON, ma il body della Response non arriva già pronto: va letto e convertito. Servono due passaggi asincroni, quindi due await: uno per aspettare la risposta HTTP (fetch), uno per aspettare che il body venga letto e trasformato da testo JSON a oggetto JavaScript (.json()).

```
const response = await fetch(url); // 1° await -> Response
const data = await response.json(); // 2° await -> dati veri
console.log(data);
```

7. GET vs POST

GET si usa per chiedere/leggere dati, senza modificare nulla sul server: è il metodo di default di fetch, basta fetch(url). POST si usa per mandare dati al server (creare, salvare, cercare in modo complesso): servono method 'POST', headers (di solito Content-Type application/json) e un body con i dati convertiti tramite JSON.stringify().

```
// GET
const r1 = await fetch('https://api.esempio.it/utenti');

// POST
const r2 = await fetch('https://api.esempio.it/utenti', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nome: 'Paolo', cognome: 'Rossi' })
});
```

8. La struttura di una HTTP request

Ogni richiesta HTTP, quella che fetch() costruisce per noi, ha tre parti: la riga iniziale (request line) con metodo, URL e versione del protocollo; gli headers, coppie chiave-valore con informazioni aggiuntive (formato dati, token, cookie); il body, i dati inviati, tipico di POST/PUT. Anche la risposta ha tre parti: status line (es. 200 OK, 404 Not Found), headers e body.

```
// URL: http[s]://[sottodominio.]dominio.estensione[:porta]/percorso[?chiave1=valore1&chiave2=valore2]
// Esempio: http://www.miosito.it/utenti?nome=Mario&eta=20
```

9. URL template con .replace()

Spesso l'URL di una API ha una parte fissa e una parte che dipende da cosa cerca l'utente. Un pattern comodo è scrivere un URL 'modello' con un segnaposto, e sostituirlo con il valore vero usando .replace() prima di fare la fetch.

```
const template = 'https://api.esempio.it/dizionario/%WORD';
const parola = 'ciao';
```

```
const urlFinale = template.replace('%WORD', parola);  
// https://api.esempio.it/dizionario/ciao
```

10. Leggere la struttura di un JSON di risposta

Davanti a dati ricevuti da una API, il primo passo non è scrivere subito il codice che li usa: è stampare tutto con `console.log` per capire come è fatto l'oggetto/array ricevuto, quali chiavi ha, se contiene array annidati. Solo dopo si scrive il codice che accede ai campi giusti.

```
console.log(data);  
// { utenti: [ { id: 1, nome: 'Mario', email: '...' }, ... ], totale: 2 }  
console.log(data.utenti[0].nome); // 'Mario'  
data.utenti.forEach((u) => console.log(u.id, u.nome, u.email));
```